ITM UNIVERSITY, GWALIOR

A

MACHINE LEARNING TRAINING PBL

QUBITWATCH: PREDICTING TEMPORAL NOISE DRIFT IN QUANTUM COMPUTING SYSTEMS

Submitted to

In partial fulfillment of the requirements for

MACHINE LEARNING PRINCIPLES (AIL0303)

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

Submitted By

Devraj Singh Sengar

Enrollment No. BETN1AI25230

Under the Supervision of

Vani Agrawal

Department of Computer Science & Engineering

ITM University, Gwalior

PROJECT TYPE | MACHINE LEARNING TRAINING REPORT

Domain: Quantum calibration drift and readout-error prediction

Dataset: IBM Quantum calibration records | Project: QubitWatch

Abstract and Keywords

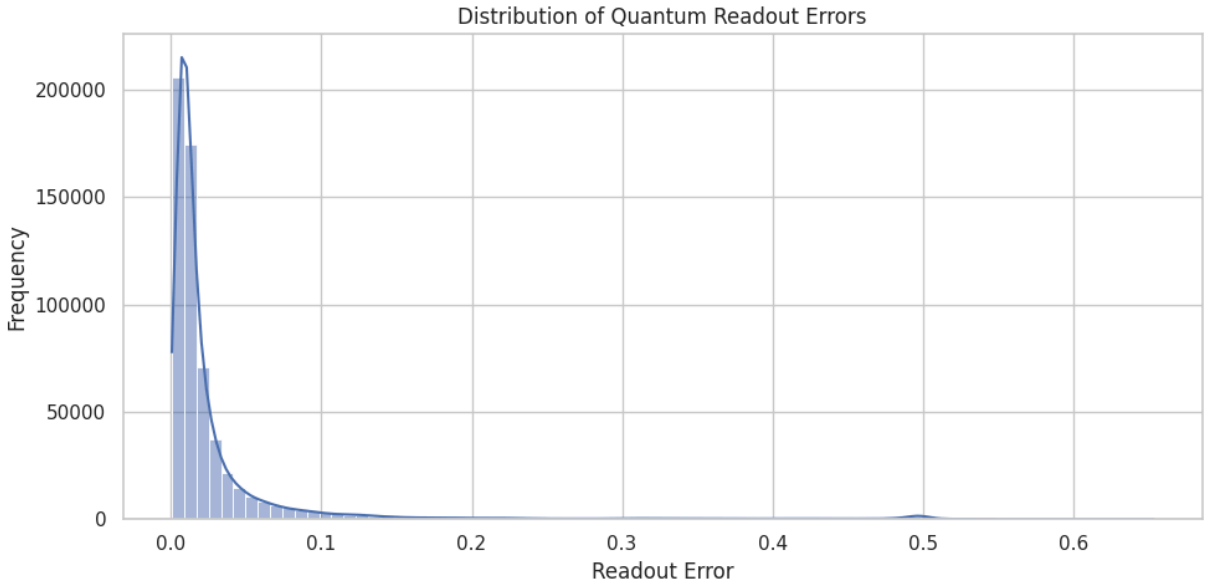
Quantum computing devices are sensitive to fluctuations in measurement behavior, including readout error. QubitWatch is a machine-learning project designed to forecast the readout error of a backend–qubit series at its next recorded IBM Quantum calibration observation. The project studies a cleaned collection of 594,198 records across four IBM backends and 601 backend–qubit time series. Historical readout measurements, lag variables, rolling statistics, elapsed-time variables and qubit identity information are used to train regression models. The experiment evaluates a persistence baseline, linear regression, LightGBM, Random Forest, tuned tree models and an LSTM using chronological partitions. In the corrected shared test evaluation of 88,478 targets, original LightGBM attained the lowest mean absolute error of 0.012556, while tuned LightGBM attained the lowest root mean squared error of 0.038324 and highest R^2 of 0.728270. The improvement over persistence is useful but modest. The output is a next-record prediction rather than a fixed-hour forecast or hardware-failure alert. The associated website presents an explanatory research demonstration.

Keywords: Quantum computing; readout error; temporal drift; regression; LightGBM; calibration data.

1. Introduction

A qubit measurement does not always return the intended state. The recorded readout-error value describes the probability of an incorrect measurement. Because calibration behavior may change over time, relying entirely on the latest measured value may miss meaningful variation. QubitWatch investigates whether recent history can predict the next recorded readout error more accurately than simply carrying forward the present value.

The practical use is exploratory monitoring and comparison of predictive methods. The model is not a live connection to quantum hardware and does not predict the exact date of the next calibration. Each forecast concerns the next observation in the recorded series, whose time interval is variable.



Notebook: distribution of observed readout errors.

2. Project Objectives

1. Construct separate chronological readout-error series for each backend and qubit.

2. Engineer a reproducible set of time-aware numerical and identity features from past and present observations.

3. Train regression models with appropriate validation, and compare them with a persistence baseline.

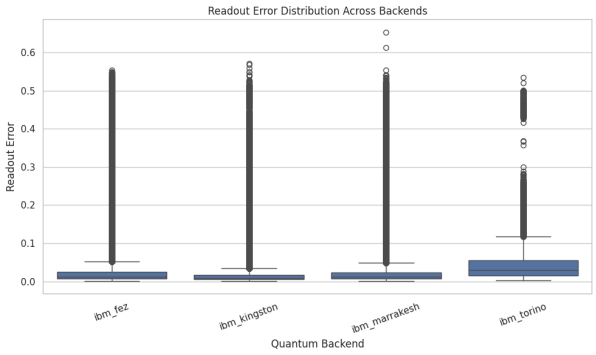
4. Evaluate models on a common held-out chronological test set using MAE, RMSE and R^2.

5. Describe the project limitations, learning outcomes and an understandable demonstration of historical data.

3. Tools and Dataset

Item	Project evidence
Data source	IBM Quantum calibration/readout observations in supplied cleaned CSV. Original collection / license conditions should be verified by the student.
Cleaned file	quantum_readout_error_cleaned (1).csv
Dataset shape	594,198 records; four backends; 601 backend–qubit series (poster and website).
Coverage	January through October 2026.
Raw fields	29 source columns including backend, qubit_a, value, observed_time and calibration metadata.
Prediction target	readout_error of the next observation in the same backend–qubit series.
Environment	Python, Google Colab/Jupyter; pandas, NumPy, scikit-learn, LightGBM, PyTorch, Matplotlib, Seaborn, joblib.

The original export also includes weather and space-weather columns, but the main shared-comparison model uses 17 recent-calibration and identity features; it does not establish that all raw columns were used as predictors. No personal records are required by this project.



Notebook: distribution across quantum backends.

4. Methodology — Data Preparation

4.1 Series reconstruction and auditing

The notebook parses mixed-format UTC timestamps and sorts rows by backend, qubit and observed time. It examines missing fields, exact duplicates, repeated timestamps, time gaps, calibration events and extreme readout-error observations. In the modelling pathway, 593,900 audited observations were retained before historical-feature filtering; 587,890 remained with sufficient history. The notebook reports 6,010 excluded rows and 601 independent series.

4.2 Temporal features

Current error; lags at 1, 2, 3 and 5 records; rolling mean and standard deviation at windows of 3, 5 and 10 records; time gap in hours; error change; error-change rate; and calibration age form the 15 numeric features. Backend and qubit identity supply two categorical features, yielding 17 model inputs.

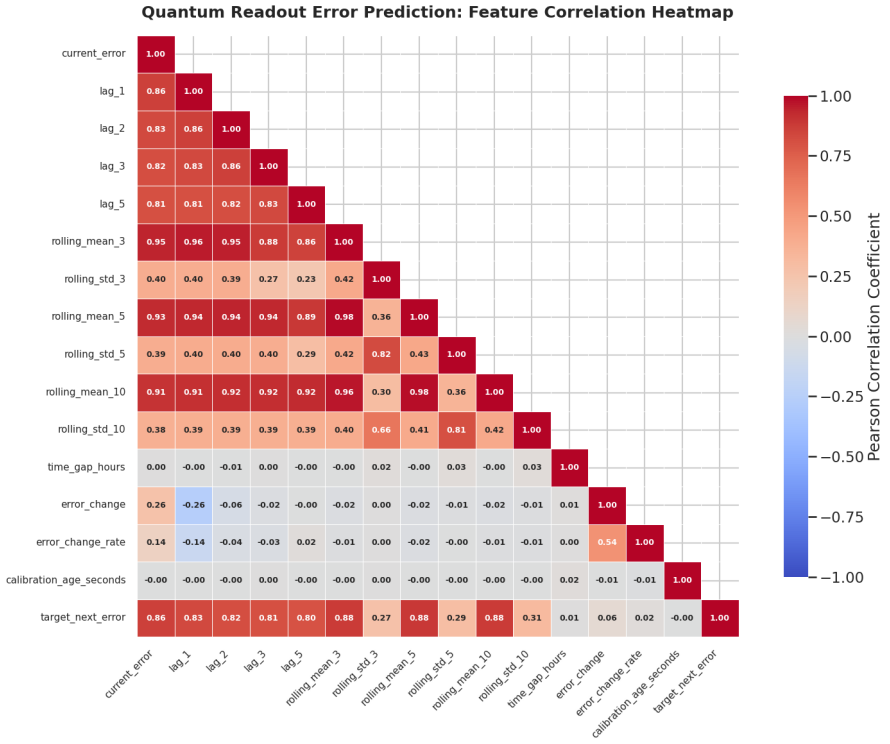
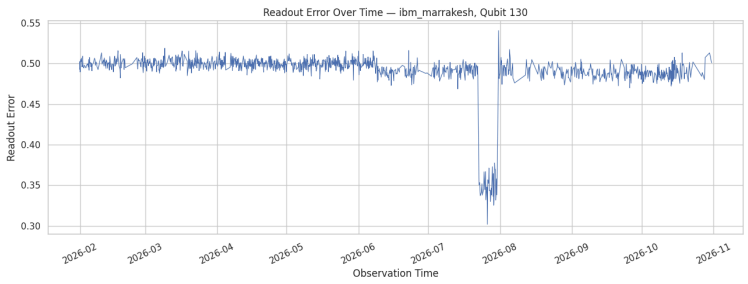


Figure 1. Pearson correlation heatmap reproduced from the uploaded training notebook (cell 79). It is an exploratory association plot, not evidence of causality.



Notebook: time variation in readout-error observations.

4. Methodology — Training and Validation

4.3 Workflow

Raw readout records → timestamp/duplicate audit → backend–qubit sorting → lag and rolling-feature creation → next-observation target → chronological train/validation/test split → baseline and model training → held-out evaluation.

4.4 Time-series split and target alignment

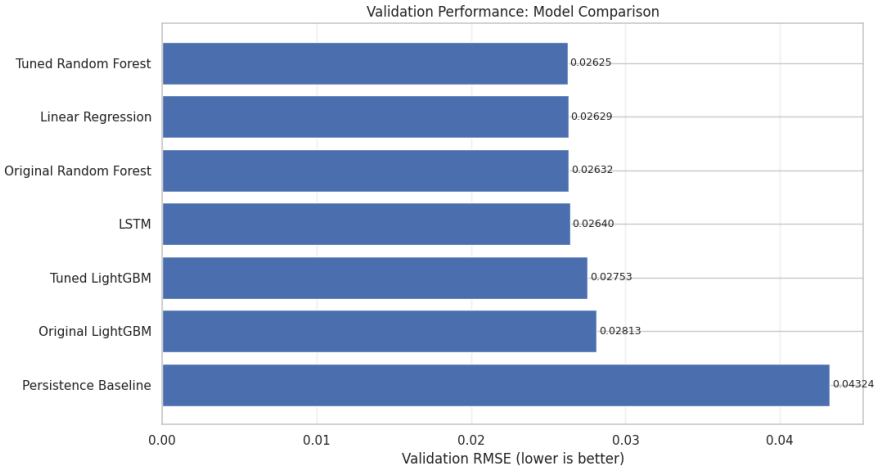
A roughly 70%/15%/15% chronological split is performed separately within each backend–qubit time series, with no random shuffling. The notebook identifies 1,202 records whose next-observation labels crossed an original split boundary (601 train-to-validation and 601 validation-to-test). A later correction assigns each prediction to the split containing its target observation and excludes the 601 terminal rows without a next target.

Corrected subset	Observations
Training	410,603
Validation	88,208
Test	88,478

All 601 series occur in all three partitions. The final comparison uses a shared test set of 88,478 next-observation targets; this corrected experiment is the primary evidence reported below.

4.5 Algorithms

Persistence directly predicts that the next error equals the latest error. The notebook also evaluates linear regression, original and tuned LightGBM, original and tuned Random Forest and a sequence-based LSTM. Model hyperparameter selection uses validation results; test results are reported separately. The poster includes a different tuned-XGBoost run that is not directly comparable with the shared experiment.



Notebook: model validation MAE comparison.

5. Implementation

5.1 Notebook code stages

The uploaded Models_training (1).ipynb is the primary code artifact. The major steps are: CSV loading and audits (cells 0–20); historical-feature construction (cell 31); time split (cell 32); persistence and model training (cells 33 onward); split-alignment audit and correction (cells 53–55); final shared-test evaluation and visualization (cells 67–79); and model export/verification (cells 80–84).

5.2 Representative source-code excerpts

Feature grouping (from notebook cell 31)

```
GROUPS = ["backend", "qubit_a"]
feature_df = model_df.sort_values(GROUPS + ["observed_time"])
grouped_error = feature_df.groupby(GROUPS)["readout_error"]
for lag in [1, 2, 3, 5]:
    feature_df[f"lag_{lag}"] = grouped_error.shift(lag)
```

Corrected target alignment (from notebook cell 55)

```
corrected_df["target_split"] = grouped["dataset_split"].shift(-1)
corrected_df = corrected_df.dropna(subset=["target_split", "target_next_error"])
corrected_df["dataset_split"] = corrected_df["target_split"]
```

5.3 Execution and artifacts

Use the provided notebook and CSV together in Google Colab. Set the input paths to the actual upload location, install dependencies and execute cells in order. Some later notebook cells load saved model files from /content; these are not included among the supplied uploads and must be regenerated or obtained from the team before replaying those specific cells. The original executed notebook outputs already document the training run.

The project web page is a frontend research demonstration. It explicitly identifies its prediction endpoint as not connected; do not present the web page as live inference.



Notebook: LSTM training and validation loss.

6. Results and Evaluation — Shared Test

The corrected common test contains 88,478 predictions across 601 series. Lower MAE and RMSE are better; higher R^2 is better. Readout error is a probability-like fraction, so MAE and RMSE are errors in the same fractional units, not percentage accuracy.

Model	MAE ↓	RMSE ↓	R^2 ↑
Persistence	0.013202	0.043243	0.654033
Original LightGBM	0.012556	0.038733	0.722432
Original Random Forest	0.012742	0.038668	0.723371
Tuned LightGBM	0.012619	0.038324	0.728270
Tuned Random Forest	0.012580	0.038338	0.728073
Linear Regression	0.013160	0.040126	0.702106
LSTM (corrected)	0.014412	0.044462	0.634253

Table 1. Final corrected shared-test measurements transcribed from the executed notebook and project poster. These are observed results, not new runs.

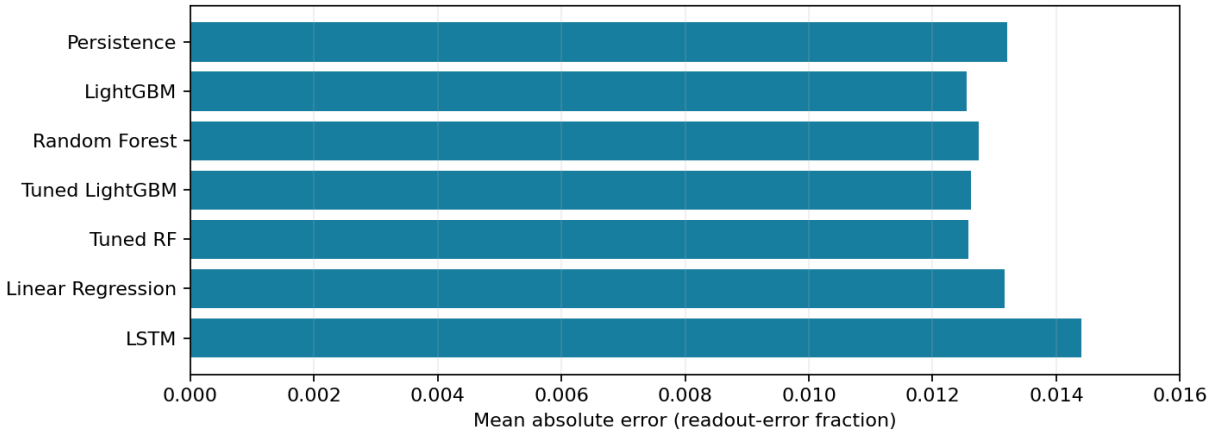
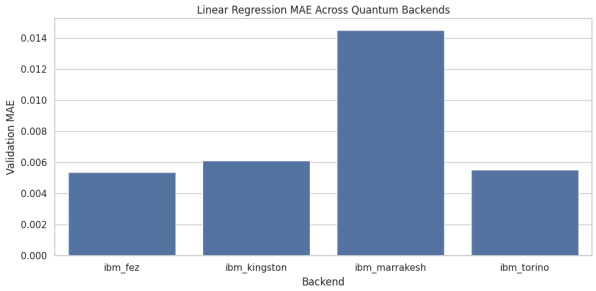


Figure 2. Comparison of mean absolute error on the common 88,478-observation test set, reconstructed from Table 1.



Notebook: per-backend linear-regression MAE.

6. Results and Evaluation — Interpretation

6.1 What the comparisons show

Original LightGBM has the lowest MAE (0.012556), approximately 4.9% below persistence MAE (0.013202). Tuned LightGBM has the lowest RMSE (0.038324) and highest R^2 (0.728270); tuned Random Forest is close behind. The LSTM in this corrected experiment performs worse than the persistence baseline. This demonstrates the value of simple baselines before interpreting more complex architecture gains.

6.2 Correlation findings

The notebook/poster reports correlations of about 0.88 between the next readout error and short rolling means, 0.86 for current error and 0.83 for lag 1. Recent error history is informative; it also explains why the persistence baseline is strong. Correlation does not establish that any feature causes the later error.

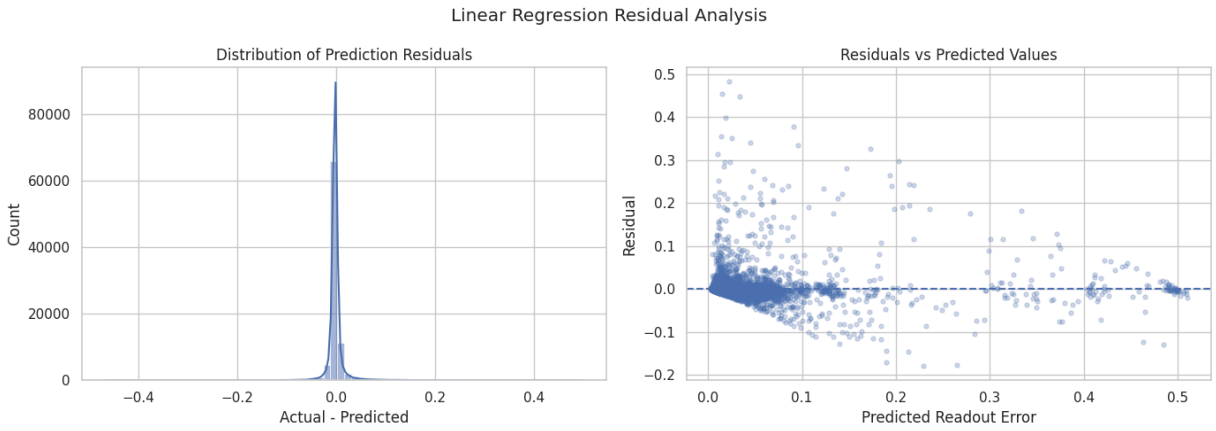
6.3 Separate tuned-XGBoost experiment

Experiment	Test size	MAE	RMSE	R^2
Tuned XGBoost	45,728	0.008530	0.024727	0.842104

Table 2. Separate XGBoost result from the QubitWatch poster and website. Its dataset version, feature set (168 one-hot features) and split differ from the shared 17-feature experiment. Consequently, it must not be ranked against the Table 1 models.

6.4 Proof of outputs

The supplied executed Jupyter notebook contains model tables, diagnostic text outputs, training logs and image outputs. The original research poster documents the same corrected test values. Per-observation actual-versus-predicted arrays were not available in the exported materials, so no such plot has been manufactured.



Notebook: linear-regression residual diagnostics.

7. Discussion, Limitations and Ethics

Project goals and use

The training work demonstrates a complete supervised regression pathway: raw calibration records, chronologically valid features, baseline comparison, multiple algorithms, error metrics and a documented model handover. The narrow result is prediction of the next recorded value for a known hardware series. Potential uses include prioritizing monitoring attention and studying temporal behavior, subject to real-world validation.

Limitations

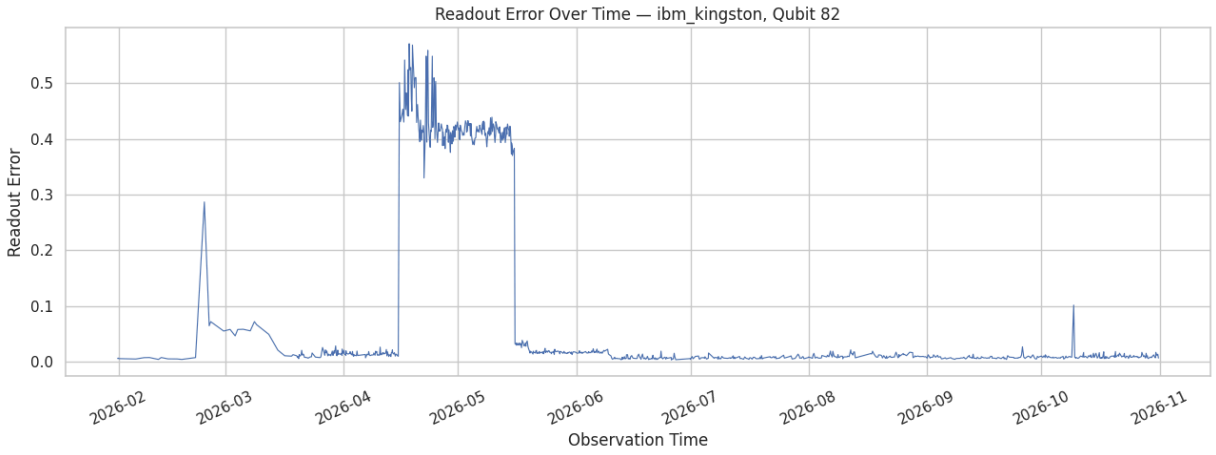
The gap between consecutive recorded observations varies; the website reports a median interval near 3.95 hours and a maximum of roughly 318 hours. Hence this is not a fixed-time forecast. The dataset contains only four backends and the historical period shown; generalization to unseen backends, other months or hardware updates is not yet established. Model accuracy can be affected by missing observations, duplicate timestamps, sampling variation and unusual error spikes. The web demonstration is frontend-only and does not call a deployed inference backend.

Practical and ethical considerations

Reported metrics must be tied to their own held-out dataset and feature definitions. Predictions must not be described as hardware-failure alarms or proof that a physical mechanism caused drift. The CSV does not require publishing personal data. Any source licensing and sharing rights should be confirmed before redistributing the data publicly, and collaborators must be credited accurately. Results should not be replaced by invented screen captures or claims of deployment.

Suggested improvements

Run controlled tests on newer time periods and unseen backends; record per-example predictions for residual and error-distribution plots; assess uncertainty; establish a functioning model inference endpoint; and evaluate how prediction quality changes with observation gap.



Notebook: observed readout error over time.

8. Conclusion and Training Learnings

QubitWatch completed a comparative machine-learning study on historical IBM Quantum readout observations. Its strongest common-test results improve modestly upon the persistence baseline, and the corrected test protocol prevents a misleading comparison across target-time boundaries. The work demonstrates that careful data alignment and transparent benchmarks can matter as much as increased model complexity.

Key learning outcomes

1. Time-series evaluation needs chronological splits and close scrutiny of label boundaries.

2. Lag and rolling-window features can capture meaningful short-term dependence.

3. MAE, RMSE and R^2 reveal different aspects of regression performance.

4. A simple persistence baseline is essential for judging incremental ML gains.

5. Experimental results from different features or splits cannot be honestly combined into a common leaderboard.

References and project evidence

[1] QubitWatch team, Models_training (1).ipynb, executed training and evaluation notebook, supplied project artifact, 2026.

[2] QubitWatch team, quantum_readout_error_cleaned (1).csv, cleaned IBM Quantum calibration data export, supplied project artifact, 2026.

[3] Team Qubits, QubitWatch Research Poster: Predicting Temporal Noise Drift in Quantum Computing Systems, supplied PDF poster, 2026.

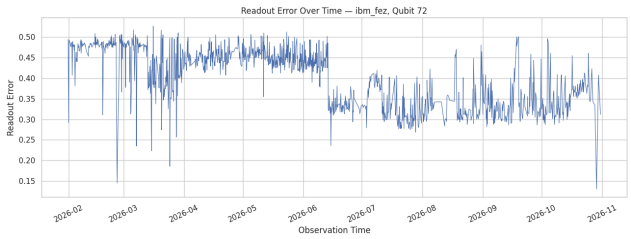
[4] QubitWatch research demo, https://quantum-drift.vercel.app/ (accessed 3 November 2026).

[5] Google Colab notebook (project link), https://colab.research.google.com/drive/1-PDxufWPwXqUXDTFmZLMKbnQvBthOIcX?usp=sharing (shared project notebook; access permissions may apply).

[6] scikit-learn documentation, https://scikit-learn.org/stable/; LightGBM documentation, https://lightgbm.readthedocs.io/; PyTorch documentation, https://pytorch.org/docs/stable/.

Appendix: accompanying submission evidence

The separate code ZIP contains the original executed notebook, cleaned dataset, execution README and selected notebook figures. A short live demonstration may show the recorded notebook outputs and website; a recording is only required if instructed by faculty.



Notebook: time series of readout errors.

Appendix A: Reproducibility and Demonstration

Submission evidence and access instructions

A.1 Dataset source and attribution

Hugging Face dataset: phanerozoic, qiskit-calibration-drift.
https://huggingface.co/datasets/phanerozoic/qiskit-calibration-drift
The dataset page states the license as CC BY 4.0. The accompanying CSV is the student-provided cleaned extract, not the entire dataset hosted upstream. The submitted data contains quantum-device calibration records rather than personal user information.

A.2 Reproducibility package

The companion ZIP includes Models_training.ipynb, quantum_readout_error_cleaned.csv, a README.md, selected original notebook figures and a demonstration checklist. The notebook records genuine model evaluation outputs. Some cells reference intermediate saved model files that were not uploaded; re-running those cells may require regenerating model artifacts.

A.3 Short demonstration sequence

1. Open the original executed notebook in Colab or Jupyter and identify the cleaned CSV.
2. Explain the target: next recorded readout error for the same backend and qubit.
3. Present lag, rolling-window and time-gap features and the chronological partition.
4. Show genuine notebook outputs and compare persistence with tree-based methods and LSTM.
5. Highlight the corrected common test of 88,478 predictions across 601 series.
6. Explain the frontend website; do not claim it has a live model inference endpoint.
7. Record a video only if the faculty explicitly requests a recorded demonstration.

A.4 Verified model findings

Best common-test MAE: original LightGBM, 0.012556 (persistence: 0.013202). Best common-test RMSE and R^2: tuned LightGBM, 0.038324 and 0.728270. The separately tuned XGBoost experiment uses a different feature set and evaluation setup, so it is not directly comparable.

A.5 Project access links

Notebook: https://colab.research.google.com/drive/1-PDxufWPwXqUXDTFmZLMKbnQvBthOIcX?usp=sharing
Project frontend: https://quantum-drift.vercel.app/
Dataset source: https://huggingface.co/datasets/phanerozoic/qiskit-calibration-drift